CS781 Capstone Project: In-House Legal Intelligence System

Sanskaar Srivastava
220967
sanskaars22@iitk.ac.in
Indian Institute of Technology Kanpur (IIT Kanpur)

Abstract

I built a legal language system from scratch on the official Indian legal corpus, with 50–300M parameters and free Kaggle GPUs. Of five pretrained language models, the best, a 101M-parameter modern decoder, reached a BERTScore of 0.8407 on passage continuation with MBR decoding. Four ≈100M models were then adapted to six tasks, from named entity recognition to prior case retrieval, with full fine-tuning, LoRA variants, LoRMA, adapters and distillation. M5's bidirectional prefix-LM pretraining was a strong starting point for the reading tasks, and task structure (CRF decoding, thresholds, retrieval normalisation) often mattered as much as the backbone. Scarce data, long documents, rare labels and metrics that do not check factual correctness remain the main limits.

1 Competition Result

Codabench username: s_220967 **Roll number:** 220967

Table 1 lists the best valid score of each phase. The template lists six phases; Phase 7 (prior case retrieval) was added to the competition later and is reported in the same way.

Table 1: Best phase results.

Phase	Task	Primary metric (value)	Rank	Valid submissions
1	Legal LM Pretraining	BERTScore F1 0.8407	14	11
2	Legal NER	strict macro-F1 0.5122	76	27
3	Rhetorical role prediction	macro-F1 0.5713	19	41
4	Court judgment prediction	macro-F1 0.7609	38	18
5	Legal statute identification	macro-F1 0.3664	80	7
6	Abstractive summarization	BERTScore F1 0.8381	68†	6
7	Prior case retrieval	Micro-F1@7 0.4318	49	8

Total valid Codabench submissions across all phases: 118 **Date ranks were recorded:** 9 November 2026

Rank = position of my best submission on the phase leaderboard. Valid = submissions that finished scoring; the course minimum is 4 in Phase 1 and 6 in Phases 2–7. †Phase 6 was still open when the rank was recorded.

2 Introduction and Problem Description

The capstone asked me to build a small legal language model from scratch, then adapt it to six Indian legal tasks from the IL-TUR benchmark [1]. The tasks cover passage continuation, named-entity recognition, rhetorical role classification, judgment prediction, statute identification, summarisation and prior-case retrieval. They use different metrics, including exact span F1, macro-F1, BERTScore and Micro-F1@7.

Models had to use 50–300M parameters and only the official data; my downstream systems had to start from my Phase 1 checkpoints. Training was limited to two Kaggle T4 GPUs in 12-hour sessions, so I compared four architectures and several adaptation methods, including the required LoRMA [2] and knowledge distillation. I did not use external data, search hyperparameters or run multiple seeds. Table 3 connects the main data findings to the decisions they informed; later sections report the experiments and their limitations.

3 Data Analysis and Preprocessing

3.1 Sources and official splits

I used only the official `Exploration-Lab/CS-781-Capstone` data. Table 2 lists the splits. For local evaluation, I held out 10 of the 70 Phase 2 training documents and 200 Phase 6 training pairs; Phase 6 has no official development split. The Phase 3 files are split by domain, and Phase 7 provides queries, relevant cases and candidate pools.

Table 2: Official splits (q = queries, c = candidate pool); median length in words (Phase 3: per sentence; Phase 6: document → summary). †After removing the test cases contained in `dev.json`.

Phase	Train	Dev	Test	Median length	Metric
1 LM	1.3 GB SC text (of 19.2 GB)	—	1,000 prompts	512 / 128 words	BERTScore F1
2 NER	70 docs, 8,393 spans	10 docs (held out)	35 docs	4,498 words	strict macro-F1
3 RR	80 docs, 16,845 sent.	10 docs, 2,142 sent.	10 docs, 2,197 sent.	27 words/sent.	macro-F1
4 CJPE	31,677 cases	994 cases†	1,517 cases	2,030 words	macro-F1
5 LSI	42,750 docs	10,181 docs	13,019 docs	430 words	macro-F1 (100)
6 Summ.	7,030 pairs	200 held out	100 docs	3,146 → 641	BERTScore F1
7 PCR	827 q / 4,320 c	118 q / 1,023 c	237 q / 1,727 c	3,947 words	Micro-F1@7

3.2 Findings that shaped the design

Figure 1 and Table 3 summarise the data patterns that influenced my choices. I made these choices using training and validation data, not Codabench scores.

- The pretraining files cover the Supreme Court (1.3 GB), High Courts (10.7 GB), Tribunals (6.5 GB) and Acts (0.7 GB).
- Phase 2 requires exact character offsets: a token span such as *"Ghosh,"* must be trimmed to *"Ghosh"*.
- Phase 4 has 31,677 distinct training texts in 37,387 rows.
- In Phase 7, *CITATOR INFO* matched the query in 105 of 214 sampled relevant pairs, so I removed it.

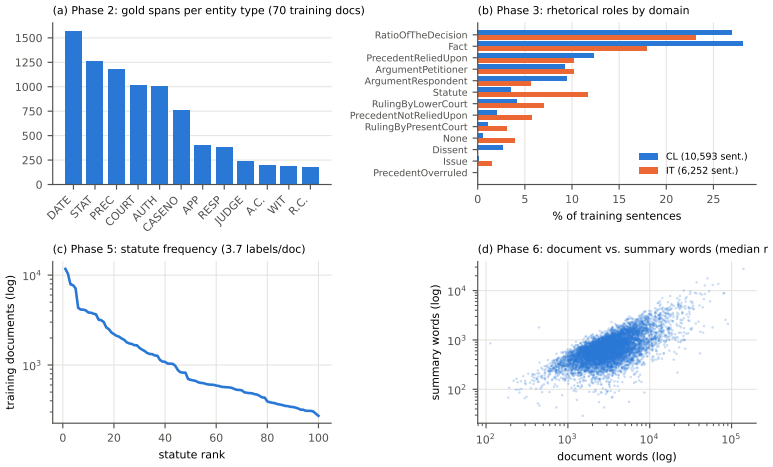


Figure 1: Training-data statistics that shaped the design (computed from the official files).

3.3 Preprocessing decisions

I used one byte-level BPE tokenizer [3] (50,001 tokens, no case or text normalisation) across phases. For Phase 2, I added character offsets without changing token IDs, then converted BIO tag runs back to spans and trimmed boundary punctuation, brackets and possessives. This raises the round-trip strict macro-F1 ceiling from 0.605 to 0.979 (Table 18). I kept the supplied Phase 3 sentences, cutting the roughly 1% longer than 256 tokens. Phase 4's `dev.json` included all 1,517 labelled test cases; I removed them, used the remaining 994 cases for validation and deduplicated training data without using test labels. I merged duplicate Phase 5 records by taking the union of their labels. In Phase 7, I removed *CITATOR INFO* and extracted each case date from its header or, if absent, from the latest year mentioned.

Table 3: Data findings and the design decisions they caused.

Ph.	Finding (training data)	Decision	Sec.
1	Test prompts closest to Supreme Court text (fertility 1.070 vs. 1.085 for a uniform mix)	Tokenizer and pretraining on that file only	4.1
1	≈200M tokens per model fit the budget (≈2 per parameter)	Fixed shape; effort spent on throughput	B
2	40% of gold spans do not end on a word boundary	Character offsets, trimming rules (ceiling 0.979)	3.3
2	One wrong tag breaks an exact span; softmax can emit invalid BIO runs	CRF output layer	4.3
2	Parties in the first 256 tokens (98%/94%); half of all mentions beyond token 1,024	Overlapping windows, 256-token preamble	4.3
2	Names repeat within a judgment	Vote and propagate rules	4.3
3	Roles follow the order of a judgment and differ between CL and IT	Document windows, sentence CRF, training Viterbi	4.4
3	Two roles absent from development (macro-F1 bound 11/13)	Ensembles rather than single-run selection	4.4
4	Median case 2,030 words; the outcome is stated at the end	Chunk reader, tail reading, section analysis	4.5
4	Labelled test cases inside `dev.json`; 41% positive in training vs. 50% in dev	Removed (994 dev cases); decision threshold	3.3
5	3.7 of 100 statutes per document; 272–11,731 documents per statute	Log-odds biases, per-statute thresholds	4.6
5	78% of documents fit in 4×512 tokens	Read the head 4×512 tokens	4.6
6	Compression 4.6; 79% of summary bigrams occur in the document; no dev split	Conditional LM on head+tail; 200 held out	4.7
7	A query is never its own answer; relevant cases are earlier	Self and date rules	4.8
7	A candidate is relevant to ≈1 query; CITATOR INFO leaks labels	Column normalisation; field deleted	4.8
7	Other queries as answers: 19.4% in training vs. 2.0% in dev	Query penalty, also every setting with p=0	4.8

4 Approach and Model Evolution

4.1 Phase-1 foundation model

I trained five models in sequence (Table 4). Model 1, a GPT-2-style decoder [4], produced loops (*"to the High Court to the High Court"*). Model 2 added rotary positions [5] and grouped-query attention [6] (12 query heads sharing 4 key/value heads) at 101M parameters; it was stopped at 74% of its epoch. Model 3, a GRU encoder–decoder with Bahdanau attention [7], fitted only 98M tokens because of its sequential recurrence. Model 4 keeps Model 2's shape but uses RMSNorm [8], SwiGLU [9], no biases, scaled residual initialisation, a tied output layer, 65k-token batches and a chunked cross-entropy that never builds the full logit tensor; it completed 214M tokens (Fig. 2). Model 5 is that network trained as a prefix-LM [10] (a 64–640-token prompt read bidirectionally, loss on the continuation only). Parameter counts were verified by summing all parameter tensors.

The sequence tests several design choices under the same compute limit: M1 was the starter baseline; M2 added RoPE and grouped-query attention; M3 tested a recurrent encoder–decoder; M4 kept M2's shape but used a faster modern training recipe; and M5 changed M4 to prefix-LM training. M4 processed the most data, while M3's recurrence limited it to 98M tokens. I carried M2–M5 into downstream tasks; M1 served only as a baseline. Appendix B records the architecture trade-offs in more detail.

Table 4: Phase 1 models (validation loss in nats/token, *continuation tokens only; LB = Codabench BERTScore with each model's original decoding). M2–M5 are adapted in Phases 2–7.

Model	Type	Shape	Position	Norm, MLP	Params	Val. loss	LB
M1	decoder (GPT-2 style)	8L, d=560, 8 heads, ctx 2048	sinusoidal	LayerNorm, GELU	58.2M	4.742	0.7830
M2	decoder	10L, d=768, GQA 12/4, ctx 1024	RoPE	LayerNorm, GELU	101.4M	3.007	0.8087
M3	GRU enc.–dec.	2L bi-GRU + 2L GRU, h=1024	recurrent	—	91.7M	4.808*	0.7874
M4	modern decoder	10L, d=768, GQA 12/4, ctx 1024	RoPE	RMSNorm, SwiGLU	101.3M	**2.992**	**0.8324**
M5	prefix-LM	as M4	RoPE	RMSNorm, SwiGLU	101.3M	—	0.8300

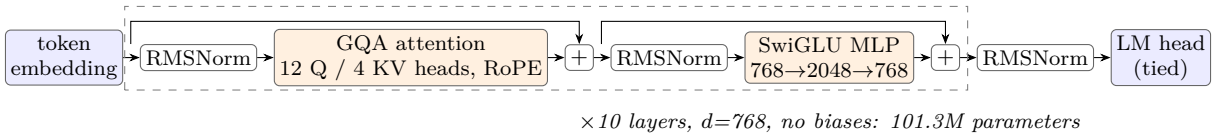


Figure 2: The Model 4 decoder (Model 5: the same network with a bidirectional prompt mask).

Decoding. With a pseudo-validation set cut from the test prompts (words 1–384 as prompt, 385–512 as reference) and a key/value-cache re-implementation of Model 4, I compared 28 settings: greedy, n-gram blocking, repetition [11] and frequency penalties, top-p [12] and min-p sampling, beam and contrastive search [13], and MBR selection [14]; pseudo-validation tracked Codabench within 0.002 (Appendix C). The best single setting decodes greedily with a 4-gram block and repetition penalty 1.2 and keeps the first 128 words (0.8363, from 0.8324). The best submission generates 14 candidates per prompt (six greedy/beam variants, eight seeded top-p/min-p samples) and returns the one with the highest mean BERTScore against the others (MBR, 0.8407).

4.2 Transfer protocol (Phases 2–7)

Each downstream model starts from M2–M5 with the tokenizer unchanged and a task-specific output layer (Fig. 3). For tagging, classification and retrieval, I removed the decoders' causal mask so each prediction could use both left and right context. M5 had already seen bidirectional prompts during pretraining; M3 used its bidirectional GRU encoder. I implemented the adaptation meth-

ods myself, including full and partial fine-tuning, LoRA, rsLoRA, LoRMA, IA^3, BitFit, Houlsby adapters and distillation. Table 5 shows which methods I tested; Appendix B.5 gives their details.

No backbone won every task. M5 led Phases 2–4 and was the strongest learned Phase 7 retriever; M4 led Phase 1 and was marginally best in Phases 5–6. M2 remained competitive on smaller Phase 4 subsets and in summarisation, while M3 generally lagged. Task-specific decisions also mattered: CRFs and training-label statistics helped sequence tasks, more data and tuned thresholds helped judgment prediction, and per-statute thresholds raised Phase 5 CV macro-F1 from 0.3546 to 0.3673. Distillation retained 99.6% and 98.0% of the teacher scores in Phases 4 and 5, but performed less well in Phases 2–3.

Table 5: Adaptation techniques per phase (models in brackets), following each phase's allowed list (Phases 5 and 7 exclude LoRA; Phase 6 excludes full fine-tuning except for the GRU), with the number of training runs.

Technique	P2 NER	P3 RR	P4 CJPE	P5 LSI	P6 Summ.	P7 PCR
Full	M3, M4	M3, M4, M5	M3, M4, M5	M3, M4, M5	M3	M3, M4, M5
Partial (top half)				M5		
LoRA	M2	M2, M4, M5	M2, M4, M5		M4	
rsLoRA		M2, M4, M5	M2, M4, M5			
LoRMA	M4, M5	M2, M4, M5	M2, M4, M5	M2, M4, M5	M5	M5
IA^3 / BitFit / Houlsby	BitFit (M3)				Houlsby (M2)	IA^3 (M4)
KD (4-layer student, teacher)	M4 (`m4_full`)	M5 (`m5_full`)	M5 (`m5_full`)	M5 (`m5_full`)	M5 (`m5_lorma`)	M5 (`m5_lorma`)
Training runs (models)	11 (M2–M5)	17 (M2–M5)	19 (M2–M5)	10 (M2–M5)	5 (M2–M5)	6 (M3–M5)*

*The two Model 2 runs of Phase 7 failed because the checkpoint dataset was not attached.

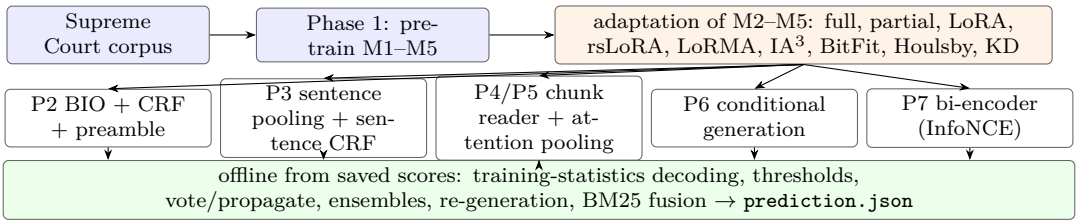


Figure 3: The system: every task model descends from my Phase 1 weights; decisions after the network are recomputed offline from saved scores.

4.3 Phase 2 — Legal NER

Because spans require exact labels and character offsets, I used BIO tagging on overlapping 1,024-token windows (stride 512) and reconstructed spans with the tokenizer's character offsets (Section 3.3). I compared softmax, BIO-constrained Viterbi and a linear-chain CRF [15, 16]. A 256-token preamble gave later windows access to the case header. At document level, *vote* assigns repeated strings their majority label and *propagate* copies a predicted label to other occurrences, following InLegalNER [17]. I dropped a greedy span ensemble because choosing it on only 10 validation documents led to overfitting.

4.4 Phase 3 — Rhetorical role prediction

I classified sentences in overlapping 1,024-token document windows so each could use its neighbours as context. Mean-pooled sentence vectors fed a linear layer or a sentence-level CRF; Corporate Law and IT data were trained together. I also decoded saved scores using training-role frequencies and transitions [18, 19], with or without a position prior, and averaged runs in ensembles.

4.5 Phase 4 — Court judgment prediction

For each case, I encoded 512-token chunks and combined them with learned attention pooling. I compared reading the head, middle and tail because outcomes usually appear near the end. Most runs used 10,000 cases; four M5 runs used all 31,677. The positive rate was 41% in training and 50% in development, so I compared argmax, training-prior and development-fitted thresholds, both per run and in ensembles.

4.6 Phase 5 — Legal statute identification

I reused the Phase 4 reader with 100 sigmoid outputs and binary cross-entropy. Since statute names are masked, the model must infer them from the facts; the first four 512-token chunks cover 78% of documents. Output biases start at the training log-odds. I compared global and per-statute thresholds, and always returned the most probable statute.

4.7 Phase 6 — Abstractive summarization

I trained conditional language models to continue each document with its summary, using 1,024 document tokens from the head and tail and up to 1,024 summary tokens. The M5 prefix-LM pretraining objective matches this setup; 79% of reference summary bigrams also occur in the source document. I tested the permitted adapters and a four-layer student, plus full fine-tuning for the GRU. Generation used greedy decoding with no repeated 4-grams; I also tested a repetition penalty and consensus decoding.

4.8 Phase 7 — Prior case retrieval

The test set has 237 queries and 1,727 candidates. I encoded each case once with a bi-encoder [20] and trained it with InfoNCE [21] and BM25 [22] hard negatives, then fused its cosine scores with BM25. Date and self-match rules remove impossible answers. Development data set the fusion weights and query penalty; column normalisation adjusts each candidate's score across queries. Gradient caching made training practical (Appendix J).

4.9 Final submitted system

Table 6 names the checkpoints behind each best submission; the repository notebooks are set to these runs.

Table 6: Checkpoints behind the best submission of each phase.

Phase	Checkpoint(s)	Decision layer / post-processing
1	Model 4 (`legal-modern-dec-101m`)	MBR over 14 decoded candidates, BERTScore utility
2	`m5_lorma_crf_pre` (M5, LoRMA, CRF, preamble 256)	CRF Viterbi + vote + propagate
3	`m5_{lora,rslora,lorma,full}_crf` (M5)	CRF ensemble, prior + training-transition Viterbi
4	`m5_{lora,rslora,lorma,full}_alldocs` (M5)	ensemble, last 8×512 tokens, dev-fitted threshold
5	`m4_full`, `m4_lorma` (M4)	ensemble, per-statute thresholds
6	`m4_lora` (M4, LoRA)	greedy, no-repeat 4-grams, repetition penalty 1.15
7	BM25 only (best model-based: ensemble of all six runs)	date/self rules, BM25 fusion, column normalisation, p=0

5 Experiments and Implementation Details

5.1 Compute and training protocol

Training ran on Kaggle (two T4 GPUs); a laptop RTX 5050 served only inference and post-processing. Phase 1 used the Hugging Face `Trainer` [23]; each of Phases 2–7 has one PyTorch [24] notebook with all its experiments, training one run per GPU in parallel and writing logs, metrics, saved scores and submissions after every run (a killed session loses at most one run); a 12-hour session held 5–8 runs of 6–192 minutes. Settings were untuned starting values shared within a phase (Table 7): AdamW, linear warm-up and decay, weight decay 0.01, clipping 1.0, token-budget batches, bf16 autocast (Phases 2–5) or fp16 with a gradient scaler (6–7); the kept epoch has the best validation metric (Phase 6: loss).

Table 7: Hyperparameters of the final model in each phase (tok. = tokens per optimizer step).

Hyperparameter	P1	P2	P3	P4	P5	P6	P7
Parameters (M), trained	101.3, all	102.5, 1.13	102.4, 1.10[a]	102.6, 1.28[a]	101.6, all[b]	102.4, 1.09	102.4, 1.11
Context length	1024	1024 windows	1024 windows	1×512 tail	4×512 head	1024+1024	2×512 head
Peak LR (backbone / adapter)	6×10^{-4}	3×10^{-5} / 3×10^{-4}	same	same	same	5×10^{-5} / 3×10^{-4}	3×10^{-5} / 3×10^{-4}
Batch × accum.	8×2 GPU×4 (65k tok.)	4 win. × 2	4k tok. × 2	4k tok. × 2	4k tok. × 2	4k tok. × 4	16 queries
Epochs (selected)	1 (3.3k steps)	8 (3)	6 (3–5)	3 (3)	3 (3)	2 (2)	6 (6)
Dropout	none	0.1	0.1	0.1	0.1	—	—
Decoding / threshold	MBR (BERTScore)	CRF + vote/prop.	prior + Viterbi	dev threshold	per statute	greedy, ng4, rp 1.15	rules + fusion

[a] LoRA member of the ensemble (also rsLoRA, LoRMA and full runs). [b] `m4_full`; its partner `m4_lorma` trains 1.38M. Warm-up 10% of steps (Phase 6: 5%); new layers use a learning rate of 10^{-3}.

5.2 Local evaluation

I evaluated on held-out data or official development sets and checked submissions with the official scorers. My Phase 3 implementation matched the official score to six decimals. For decisions fitted on development data, I also report two-fold cross-validation (CV), fitting on one half and scoring on the other across repeated splits. Local estimates matched Codabench most closely in Phases 1, 6 and 7. Phase 3 was the main exception: two roles were absent from development, lowering its macro-F1. Rescaling the 11 observed roles by 13/11 removes most of the gap. Phase 4 CV estimates were within 0.017 of test for the reported choices, except for a high-scoring run selected from about a hundred candidates.

5.3 Ablations and controlled comparisons

Table 8 changes one factor at a time on validation data; complete run tables are in the appendix.

Table 8: Controlled comparisons (one factor changed; validation scores; CV = 2-fold cross-validated).

Phase	Factor	Setting compared	Score
1	architecture (same data)	M1 → M2 → M4 (val. loss, nats/token)	4.742 → 3.007 → 2.992
1	decoding (M4 fixed)	greedy → + 3-gram block → + 4-gram, rp 1.2 → MBR (pseudo-val.)	0.8048 → 0.8351 → 0.8372 → 0.8419
2	output layer (M4 LoRMA)	softmax → constrained Viterbi → CRF	0.3202 → 0.4131 → 0.4502
2	preamble context (M4 LoRMA CRF)	none → 256 tokens	0.4502 → 0.4561
2	document rules (M4 LoRMA CRF)	none → vote / propagate / both	0.4502 → 0.4591 / 0.4739 / 0.4813
2	distillation (4-layer student)	no teacher → teacher	0.2695 → 0.2841
3	output layer (M5 LoRA)	linear → sentence CRF	0.4407 → 0.4685
3	decoding (M5 CRF ensemble)	own Viterbi → prior + training Viterbi	0.453 → 0.471
3	distillation	10-layer teacher → 4-layer student (training minutes)	0.442 (46.6) → 0.347 (17.5)
4	training data (M5 rsLoRA)	10k → 31.7k cases	0.6615 → 0.7087
4	section read (8 runs)	head / middle / head+tail / tail	0.53–0.57 / 0.50–0.58 / 0.61–0.64 / 0.62–0.66
4	distillation	teacher `m5_full` → 4-layer student (training minutes)	0.6528 (69) → 0.6503 (26)
5	threshold (M4 ensemble, CV)	one global → per statute	0.3546 → 0.3673
5	method (M5, global threshold)	full / LoRMA / partial / KD student	0.3532 / 0.3292 / 0.3364 / 0.3461
6	document part read (M5 LoRMA)	head 1024 → head+tail 1024 (BERTScore)	0.8337 → 0.8361
6	distillation	teacher `m5_lorma` → 4-layer student	0.8361 → 0.8232
7	training (M5 LoRMA, raw ranking)	Phase 1 model (0-shot) → fine-tuned	0.2424 → 0.3091
7	post-processing (M5 LoRMA)	raw → + BM25, no column term → full (CV)	0.3091 → 0.3413 → 0.4308

6 Results

Table 9 compares local and Codabench scores; full run tables are in Appendices C–I. Figure 4 shows selected learning curves. Macro-F1 also hides large class differences: the best Phase 2 F1 ranges from 0.11 (appellant) to 0.93 (date), and Phase 3 from 0.27 (*None*) to 0.82 (*Statute*). Phase 5 statute F1 ranges from 0.05 to 0.83 and rises with training frequency ($\rho = 0.57$); the rarest quarter averages 0.25, against 0.51 for the most frequent.

Table 9: Best submission per phase and model: its local score (validation; 2-fold CV where a decision was fitted on the validation data) / Codabench; a single value means the run was not scored on Codabench; — = no run.

Phase	Metric	M2	M3	M4	M5
1	BERTScore F1	— / 0.8087	— / 0.7874	0.8419 / **0.8407**	— / 0.8300
2	strict macro-F1	0.2899 / 0.3416	0.1906	0.4561† / 0.4754	0.4797† / **0.5122**
3	macro-F1	0.439 / 0.4952	0.350 / 0.3338	0.441 / 0.5245	0.471 / **0.5713**
4	macro-F1	0.7188‡ / 0.6889	0.5927	0.6787	0.7521 / **0.7609**
5	macro-F1	0.3387	0.2446	0.3673 / **0.3664**	0.3624 / 0.3588
6	BERTScore F1	0.8377 / 0.8329	0.8033	0.8423 / **0.8381**	0.8361 / 0.8322
7	Micro-F1@7	—	0.4263	0.4280 / 0.4253	0.4317 / 0.4300

†before vote/propagate. ‡fixed rule (no CV needed). Phase 7: BM25 with the same post-processing 0.4249 (CV) / **0.4318**; six-run ensemble 0.4245 / 0.4318 (both p=0).

There was no single winning backbone across all seven tasks. M5 with a four-run CRF ensemble and training-prior/transition Viterbi decoding led Phase 3 at 0.5713; M5 ensembles with more training cases and longer tail reading led Phase 4 at 0.7609; and M4 was strongest in pretraining, statute identification and summarisation. In retrieval, task-aware post-processing was decisive: BM25 with date/self rules and cross-query column normalisation reached 0.4318, matching the six-run ensemble. In these experiments, data access and task-specific decision structure often mattered more than adding model capacity.

7 Error Analysis and Discussion

I rely most on controlled comparisons and cross-validation. Codabench scores show how well local estimates transferred to test; they are not used to select methods. Validation is especially small in Phases 2, 3 and 7, so differences below about 0.01 are meaningful only when they recur across runs.

7.1 Phase-wise failure modes

Phase 1: repetition. Plain greedy decoding repeats 69% of word 4-grams. A 4-gram block and repetition penalty raise distinct bigrams from 0.27 to 0.93, but ROUGE-L falls from 0.1995 to 0.1865. The model becomes less repetitive, not better informed.

Phase 2: role-defined entities and boundaries. Appellants, respondents, counsel and witnesses reach only 0.11–0.24 F1; dates and statutes reach 0.93 and 0.85. The weak labels are all person names, often identified by a role in the header rather than near later mentions. Half of all mentions lie beyond token 1,024. Exact offsets also matter: trimming spans raises the round-trip ceiling from 0.605 to 0.979.

Phase 3: speaker and rhetorical function. The main confusions are petitioner arguments labelled as respondent arguments (136 development sentences) and facts labelled as reasoning (75).

The *None* class reaches 0.27 F1; Corporate Law is harder than IT across all 17 runs (0.362 vs. 0.465).

Phase 4: prior shift and less explicit outcomes. Acceptance is 41.4% of training labels and 50% of development labels, so models under-predict it unless the threshold is adjusted. Explicit outcome words also occur less often in test tails (13%) than in training tails (24%), making the test cases harder for models that rely on the disposition sentence.

Phase 5: rare statutes. Two statutes remain below 0.1 F1. The rarest quarter averages 0.25, against 0.51 for the most frequent. Since macro-F1 weights each label equally, the tuned thresholds favour recall; the system predicts 5.2 statutes per development document, compared with 3.9 gold.

Phase 6: short summaries and truncation. Submitted M4 summaries average 535 words, against 852 for the references; 11% reach the length cap. BERTScore compares only about the first 400 words, so a good score says little about later content or factual accuracy.

Phase 7: fixed-size retrieval and generic candidates. Returning exactly seven cases caps a perfect ranker at 0.618 on development because relevance counts vary. Generic candidates that score well for many queries are another problem; column normalisation reduces this hub effect. Post-processed systems score 0.426–0.433.

7.2 What the metrics do not measure

Strict span F1 penalises even a one-character offset error. Macro-F1 gives rare labels the same weight as common ones, so thresholds matter. Truncated BERTScore rewards similarity in the opening of a summary but does not check whether later statements are true. Micro-F1@7 always requires seven answers, regardless of how many are relevant. None of these metrics measures confidence calibration, which a lawyer would need to judge whether to trust a result.

7.3 Limitations and what I would do next

Several limits affect these results. Phases 2–3 validate on 10 documents and Phase 7 on 118 queries; each run used one seed, and hyperparameters were not tuned. The models saw about two training tokens per parameter and read at most 1,024 tokens, while many judgments are much longer. Phase 1's pseudo-validation prompts came from the test prompts, Phase 6 has no official development set, the Phase 7 M2 runs did not complete, and no legal expert reviewed the outputs.

Two roles are missing from Phase 3 development. Since the scorer averages all 13 roles and assigns zero to absent ones, its development score is reduced by the factor 11/13. Rescaling by 13/11 explains most of the 0.05–0.10 Codabench advantage seen in nine of ten submissions. In Phase 4, CV estimates were within 0.017 of test except for `m2_lorma`, whose development score (0.7188) had been selected from about a hundred candidates and fell to 0.6889 on test.

My next steps would be to run three seeds per phase, use the unused High Court and Tribunal data for continued pretraining, test bidirectional continuation of M4, tune LoRMA's learning rate against LoRA, and extend context beyond 1,024 tokens. Phase-specific tests include a sentence-only Phase 3 run, Phase 7 cross-encoder reranking, and summary length control.

7.4 Surprising findings

What I found interesting was that M5 scored below M4 as a language model but performed better in the reading tasks submitted by both. In my opinion, this shows that language-model performance alone did not predict which model would transfer better to these reading tasks. I was also surprised that, in Phase 3, role priors and training transitions raised the score from 0.504 to 0.571. In Phase 7, BM25 with post-processing matched the six-run ensemble at 0.4318; I found it notable that this

simpler approach matched the ensemble. A four-layer student also retained 99.6% of its Phase 4 teacher's score while taking 38% as long to train, which I found encouraging.

8 Conclusion

I built a compact legal-language system from scratch using the official data. With roughly two training tokens per parameter and documents longer than the 1,024-token context, data and context were bigger constraints than model size. The strongest gains often came from task-specific methods: CRFs and training statistics for sequences, thresholds for imbalanced labels, and normalisation for retrieval. M5's bidirectional pretraining also transferred well to reading tasks.

In my opinion, local scores were useful when based on held-out data or cross-validation; the largest test gaps had identifiable causes. Still, rare entity and statute predictions remain unreliable, and a high truncated BERTScore does not guarantee a factually correct summary. No legal expert reviewed the outputs, so the system should be treated as decision support, not legal advice. Code and documentation: https://github.com/D3bug1t/LLM-Pretraining-PostTraining-Extrinsic-Instrinsic-

LLM Usage Declaration

Completing this section is mandatory. Inconsistent answers relative to the code and the prose will be treated as an integrity issue.

Q1. Which Large Language Model(s), if any, did you use while completing the competition or writing this report? Give model name and vendor (e.g., none / ChatGPT-4.x / Gemini x / Grok / Copilot / other).

Answer: GitHub Copilot in VS Code assisted with understanding concepts, high-level ideation, and editing this report. Claude assisted with plotting syntax and small bug fixes. Neither was part of the competition system or used to produce the experimental results.

Q2. For each row, mark Yes or No and state the purpose in one short phrase. "Yes" is acceptable only for the uses permitted by the capstone document.

Part	Used LLM?	Purpose of using the LLM
Understanding concepts / papers	Yes	Explained modeling concepts and paper details
Ideation / brainstorming at a high level	Yes	Suggested alternative architectural choices
Debugging small ancillary snippets	Yes	Resolved PyTorch syntax errors in snippets
Plotting / visualization code	Yes	Helped write Matplotlib plotting and visualization code
Grammar / writing style of the report	Yes	Edited wording for grammar and readability
Report structure / section wording	Yes	Suggested report organization and section wording
Designing the model architecture	Yes	Identified research papers describing SwiGLU
Hyperparameter search / tuning	No	No LLM-based tuning; settings were not searched
Analysis of results / conclusions	No	–
Writing the core training / inference code	Yes	Suggested bug fixes and logical corrections
Other (specify)	No	–

Q3. Did you refer to sources other than lecture slides and an LLM? If yes, list them (papers, blogs, GitHub repositories, textbooks, official starter notebook, evaluation scripts). These must also appear in the bibliography when they influenced the method.

Answer: Yes. I used the papers cited in the bibliography, the official course starter code and data, and the official evaluation scripts for scoring and format validation.

Q4. Apart from any permitted LLM use, what was *your* contribution? In 8–12 sentences describe your own understanding, architecture decisions, implementation, experiments, and analysis. Name at least two things you tried that failed and what you concluded from them.

I designed and trained five language models from scratch on the official Supreme Court corpus,

comparing decoder architectures and a prefix-LM under the project compute limits. I built a byte-level BPE tokenizer and an offset-preserving version for exact named-entity spans. For Phases 2–7, I adapted the pretrained checkpoints with full fine-tuning, parameter-efficient methods, task-specific heads and distillation, then evaluated the runs on held-out or official development data. I implemented task-specific post-processing, including CRF and training-statistic decoding, threshold selection, document rules, and lexical/dense retrieval fusion. I also checked submissions with the official evaluation scripts and investigated gaps between local and Codabench scores. In Phase 1, MBR selection improved pseudo-validation and gave my best submission (0.8407 on Codabench, from 0.8363 for the best single decoding setting). In Phase 4, head-only reading was weaker than reading the tail, consistent with the decision appearing near the end of judgments. The four-layer student was substantially weaker than its teacher in Phases 2–3, though it retained nearly all the teacher's Phase 4 score with much less training time. I drew the experimental results and conclusions from my own runs.

References

[1] A. Joshi, S. Paul, A. Sharma, P. Goyal, S. Ghosh, and A. Modi, "IL-TUR: Benchmark for Indian legal text understanding and reasoning," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024.

[2] H. Bihany, S. Patel, and A. Modi, "LoRMA: Low-rank multiplicative adaptation for LLMs," in *Findings of the Association for Computational Linguistics: ACL 2025*, 2025.

[3] R. Sennrich, B. Haddow, and A. Birch, "Neural machine translation of rare words with subword units," in *Proceedings of ACL*, 2016.

[4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Technical Report*, 2019.

[5] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "RoFormer: Enhanced transformer with rotary position embedding," *arXiv preprint arXiv:2104.09864*, 2021.

[6] J. Ainslie, J. Lee-Thorp, M. de Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, "GQA: Training generalized multi-query transformer models from multi-head checkpoints," in *Proceedings of EMNLP*, 2023.

[7] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," in *Proceedings of ICLR*, 2015.

[8] B. Zhang and R. Sennrich, "Root mean square layer normalization," in *Advances in Neural Information Processing Systems*, 2019.

[9] N. Shazeer, "GLU variants improve transformer," *arXiv preprint arXiv:2002.05202*, 2020.

[10] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, 2020.

[11] N. S. Keskar, B. McCann, L. R. Varshney, C. Xiong, and R. Socher, "CTRL: A conditional transformer language model for controllable generation," *arXiv preprint arXiv:1909.05858*, 2019.

[12] A. Holtzman, J. Buys, L. Du, M. Forbes, and Y. Choi, "The curious case of neural text degeneration," in *Proceedings of ICLR*, 2020.

[13] Y. Su, T. Lan, Y. Wang, D. Yogatama, L. Kong, and N. Collier, "A contrastive framework for neural text generation," in *Advances in Neural Information Processing Systems*, 2022.

[14] S. Kumar and W. Byrne, "Minimum Bayes-risk decoding for statistical machine translation," in *Proceedings of HLT-NAACL*, 2004.

[15] J. Lafferty, A. McCallum, and F. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in *Proceedings of ICML*, 2001.

[16] G. Lample, M. Ballesteros, S. Subramanian, K. Kawakami, and C. Dyer, "Neural architectures for named entity recognition," in *Proceedings of NAACL*, 2016.

[17] P. Kalamkar, A. Agarwal, A. Tiwari, S. Gupta, S. Karn, and V. Raghavan, "Named entity recognition in Indian court judgments," in *Proceedings of the Natural Legal Language Processing Workshop*, 2022.

[18] A. K. Menon, S. Jayasumana, A. S. Rawat, H. Jain, A. Veit, and S. Kumar, "Long-tail learning via logit adjustment," in *Proceedings of ICLR*, 2021.

[19] H. Bourlard and N. Morgan, *Connectionist Speech Recognition: A Hybrid Approach*. Kluwer, 1994.

[20] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of EMNLP*, 2020.

[21] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[22] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, 2009.

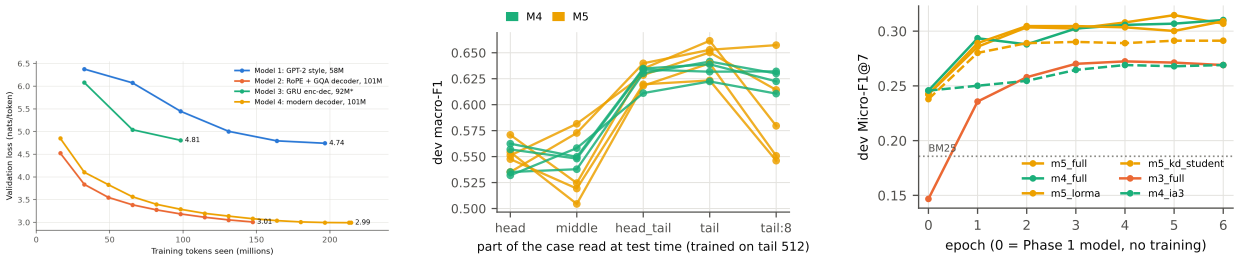
[23] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proceedings of EMNLP: System Demonstrations*, 2020.

[24] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems*, 2019.

Appendix

The appendix holds the complete per-phase tables referenced from the main text.

A Learning curves and per-class breakdowns



(a) Phase 1 validation loss vs. training tokens.

(b) Phase 4: the eight 10k-case M4/M5 runs (trained on the last 512 tokens) reading other parts at test time.

(c) Phase 7 dev Micro-F1@7 per epoch (0 = untrained Phase 1 model; dashed: KD, IA^3).

Figure 4: Learning curves and the Phase 4 reading analysis.

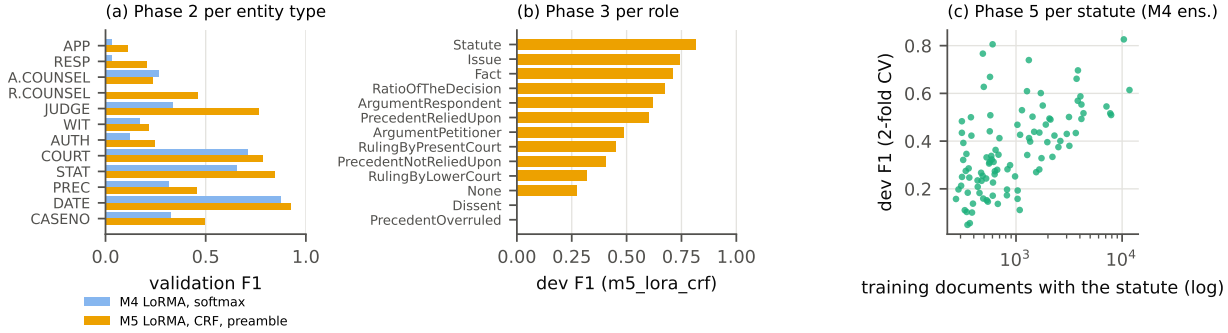


Figure 5: Per-class breakdowns: (a) Phase 2 validation F1 per entity type, first and best system; (b) Phase 3 dev F1 per role of `m5_lora_crf` (*Dissent* and *PrecedentOverruled* do not occur in dev); (c) Phase 5 dev F1 per statute against its training frequency (M4 ensemble, per-statute thresholds fitted on the other half of dev).

B Architecture and design rationale

This appendix summarises the architecture and task-layer trade-offs behind the experiments.

B.1 The constraints that shaped the choices

The model budget was 50–300M parameters; I chose about 100M so models could be trained on two T4 GPUs within a 12-hour session. The Supreme Court corpus contains about 275M tokens, or roughly two tokens per parameter for one epoch, so throughput mattered. Phase 1 required continuation of a 512-word passage, while later tasks required the same weights to handle tagging, classification, generation and retrieval on much longer documents. I designed the model sequence to test these trade-offs (Section 4.1).

B.2 The five Phase-1 models

Table 10: Phase-1 architectures: the question each one asked, its advantages and drawbacks, and the outcome (validation loss in nats/token / Codabench BERTScore; downstream results from Table 9).

Model	Question asked	Advantages	Drawbacks	Outcome
M1 GPT-2 style, 58M	Does the starter pipeline work end to end, and where does it fail?	Simple, known to train stably; 2,048-token context; quick to build.	Absolute sinusoidal positions; full multi-head KV cache; biases and LayerNorm/GELU; only 58M of the 100M budget; no repetition control.	4.742 / 0.7830. Loops such as "to the High Court to the High Court". Not carried forward.
M2 RoPE + GQA, 101M	Do relative positions, shared key/value heads and the full parameter budget help?	RoPE encodes the offset between tokens, which suits a passage starting at an arbitrary point; GQA stores 3× fewer keys/values, so decoding is faster.	Still LayerNorm/GELU with biases; small batches (16k tokens); stopped at 74% of the epoch (147M tokens).	3.007 / 0.8087. Downstream: best single 10k-case run in Phase 4 (0.6831); close second in Phase 6.
M3 GRU enc.–dec., 92M	Can the encoder–decoder family, with a bidirectional encoder and attention, compete at the same size?	The encoder reads the whole prompt in both directions; constant memory per generated token; well understood (Assignment 2).	The recurrence runs one time step at a time, so it trains slowly (98M tokens in budget); a fixed-size state summarises the context; more than half of the encoder is embeddings; no linear layers, so LoRA-type methods do not apply.	4.808* / 0.7874. Weakest model in every downstream phase (e.g. Phase 5 dev 0.2105 vs. 0.3535).
M4 modern decoder, 101M	With M2's shape fixed, does the current open-model recipe give a better model from the same budget?	RMSNorm and SwiGLU give more quality per parameter; no biases; scaled residual initialisation allows a higher learning rate; 4× larger batches, made possible by a chunked loss that saves memory, raise throughput, so it completed a full epoch (214M tokens).	Causal attention only, so reading tasks need a mask change at fine-tuning; context 1,024 tokens.	**2.992 / 0.8324** (0.8407 with MBR decoding). Best language model; best in Phases 1, 5 and 6.
M5 prefix-LM, 101M	With M4's network fixed, does reading the prompt bidirectionally during pretraining help?	Pretraining matches the downstream pattern "read a document whole, then decide or write"; less mismatch when the causal mask is removed for fine-tuning.	Loss only on the continuation (≈65% of tokens supervised); needs an explicit mask per example, so the fused causal attention kernel cannot be used.	— / 0.8300, slightly below M4 as a language model. Best in Phases 2, 3 and 4; best model-based in Phase 7.

*Loss on the continuation only; not comparable with the decoders.

What the model comparisons show. M3 processed 98M tokens, less than half of M4's 214M, because its GRU runs sequentially. This likely contributed to its weaker results; its lack of linear layers also ruled out the adapter methods used with the decoders. The M1-to-M2 comparison changes several factors at once, so it cannot isolate the effect of RoPE or GQA. M2 and M4 have the same shape, but M4 processed 45% more tokens; its better loss and Codabench score reflect both the recipe and the additional training it enabled.

M5's Phase 1 score was slightly below M4's, but it led Phases 2–4 and the learned Phase 7 retriever. One possible reason is that M5's bidirectional prompt training better matches downstream reading tasks. This was not tested directly. Where M4 led, the margins were small: 0.0004 for Phase 5 full fine-tuning and 0.0017 in held-out Phase 6 BERTScore.

B.3 Attention mechanisms

Table 11: Attention used by each Phase-1 model. KV cache = numbers stored per generated token over all layers.

Model	Mechanism	Q / KV heads	d_h	Mask	Positions	KV cache
M1	multi-head self-attention	8 / 8	70	causal	sinusoidal, added to embeddings	8,960
M2	grouped-query self-attention	12 / 4	64	causal	RoPE on q, k	5,120
M3	Bahdanau additive cross-attention (no self-attention)	1 (dim 256)	—	prompt padding	GRU recurrence	—
M4	grouped-query self-attention	12 / 4	64	causal	RoPE on q, k	5,120
M5	grouped-query self-attention	12 / 4	64	bidirectional over prompt, causal after	RoPE on q, k	5,120

GQA lets query head i use key/value group $\lfloor i/3 \rfloor$. This keeps 12 query views and cuts the cache by 3× against 12-head attention, which kept the cache small for the 28-setting decoding study and the MBR candidate pools on a laptop GPU. RoPE rotates query and key pairs by angles proportional to position, so $q_m^\top k_n$ depends only on $m - n$. With left-padded batches, each row's positions start at its first real token. In Phase 6 the same rotation was extended from 1,024 to 2,048 positions without retraining from scratch, which a learned table of absolute positions would not allow.

B.4 Task layers: what I chose and what I did not

Table 12: Task-layer choices per phase, the alternatives considered, the reasons, and the known weaknesses.

Phase	Chosen	Alternatives	Why chosen	Weakness
2	BIO tagging with a CRF on overlapping 1,024-token windows, 256-token preamble	softmax tagging; constrained Viterbi; span enumeration	The CRF scores whole tag sequences, so spans are valid and transitions are learned; the parallel partition function made it as cheap as softmax (0.036 s per batch); span enumeration over 4,498-word documents is costly	A boundary error still costs the whole span; rare party labels stay weak (Fig. 5a)
3	Mean-pooled sentence vectors from document windows, linear or sentence-CRF output	one sentence per input; a second hierarchical encoder	Neighbouring sentences are visible to the encoder; the CRF models role order; no extra network to train	Only 10 dev documents to compare variants
4	K chunks of 512 tokens, attention pooling, section chosen by experiment	truncation to the first 512 tokens; reading the whole case	Cost grows linearly with K; the section can be changed at test time without retraining	Chunks do not see each other; long cases are read only in part
5	Same reader, 100 sigmoids with log-odds bias, per-statute thresholds	softmax over label sets; one classifier per statute	One shared encoder for all labels; rare statutes start calibrated	Rare statutes remain hard (F1 0.25 in the rarest quarter)
6	Decoder as conditional LM on `[doc] <s> [summary]`	encoder–decoder (only M3); extractive selection	Reuses the pretraining objective directly, with no new parameters except the adapter	Reads 1,024 tokens of a median 3,146-word document; positions extended beyond pretraining
7	Bi-encoder, InfoNCE with BM25 hard negatives, fused with BM25	cross-encoder re-ranking; BM25 only	One vector per case scores all 409k test pairs cheaply; BM25 adds exact lexical matches	Reads only the first 2×512 tokens; 827 training queries

B.5 Adaptation techniques: advantages, drawbacks and my evidence

Table 13: Adaptation techniques (parameters trained for a 101M decoder), with evidence from my runs.

Technique	What is trained	Advantages	Drawbacks	Evidence from my runs
Full	all weights (101M)	Most capacity; reference point	Memory for optimizer state; may forget pretraining on small data	Best single run in Phase 5 (M4 0.3535, M5 0.3531) and Phase 7 raw (M5 0.3146)
Partial	top 5 of 10 layers (31.7M)	Keeps low layers as pretrained; cheaper	Lower layers cannot adapt	Phase 5: 0.3364, between LoRMA (0.3292) and full (0.3531) on M5
LoRA	$W_0x + \frac{\alpha}{r}BAx$, r=8 ($\approx$1.1M)	1% of the parameters; base weights shared	Scale α/r shrinks the update as r grows	Within 0.01–0.02 of full in Phases 3–4 (e.g. M5 Phase 3: 0.441 vs. 0.442)
rsLoRA	as LoRA, scale $\alpha/\sqrt{r}$	Update size stable in r	One more scale choice	Higher than LoRA in 5 of 8 paired runs (Phases 3–4); best Phase 4 single run (0.7087)
LoRMA	$(I + \frac{\alpha}{r}BA)W_0x$ ($\approx$1.1M)	Re-mixes the pretrained layer's outputs; same cost as LoRA via $z + \frac{\alpha}{r}B(Az)$	The update is tied to W_0's output space	Lowest of the low-rank methods on M5 in Phases 3–4 and below full in Phase 5; still the best Phase 2 submission (M5 LoRMA + CRF, 0.5122)
IA^3	per-channel scales (0.03M)	Smallest update	Little capacity	Phase 7: 0.2690 vs. 0.3102 for full M4
BitFit	biases (0.08M on M3)	Tiny; applies to the GRU	Little capacity	Phase 2 M3: 0.1149 vs. 0.1906 full
Houlsby	bottleneck layers 768$\rightarrow$64$\rightarrow$768 (2.0M)	New capacity in series	Adds inference layers	Phase 6: 0.8377 held-out, within 0.0001 of M4 LoRA
KD	4-layer student (63.6M) on $(1-\lambda)\mathcal{L} + \lambda T^2$KL	$\approx$2.6$\times$ faster training	Needs a teacher run first; loses depth	Student/teacher: Phase 4 99.6%, Phase 5 98.0%, Phase 7 94%, Phase 2 82%, Phase 3 78% of the teacher's score; Phase 2 student beats its no-teacher control (0.2841 vs. 0.2695)

C Phase 1 details

Table 14: Phase 1 decoding on Model 4: pseudo-validation vs. Codabench BERTScore for the submitted settings.

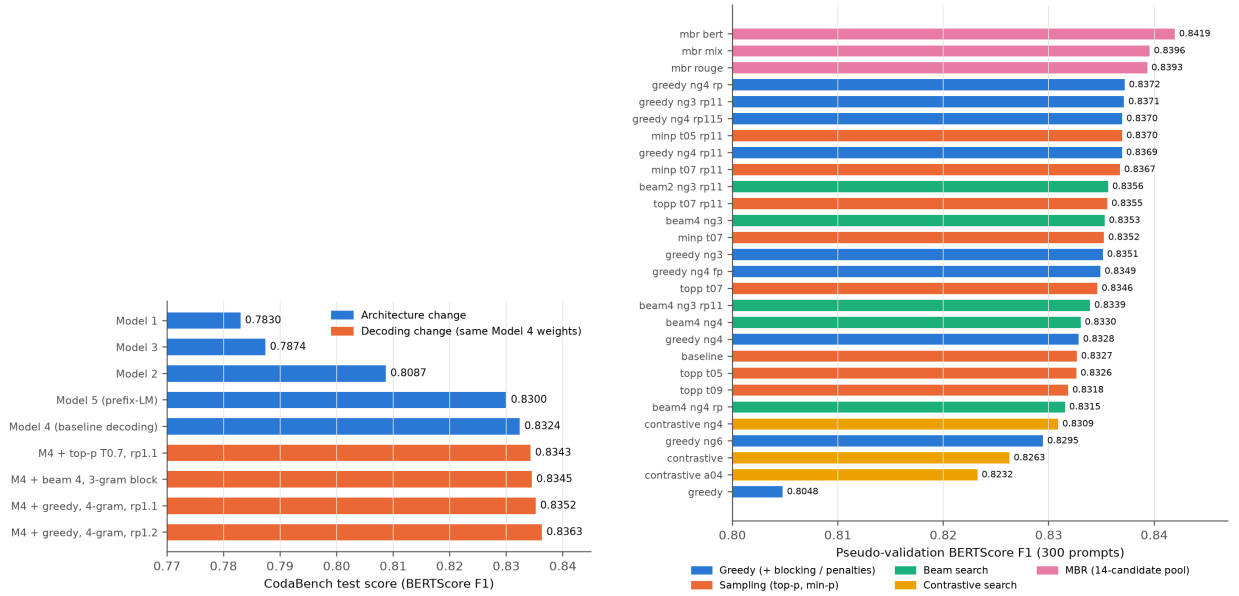
Decoding	Pseudo-val	Codabench	Δ
T=0.1, top-p 0.95, 4-gram block (notebook default)	0.8327	0.8324	-0.0003
Top-p (T=0.7, p=0.9), 3-gram, rp 1.1	0.8355	0.8343	-0.0012
Beam 4, 3-gram block	0.8353	0.8345	-0.0008
Greedy, 4-gram block, rp 1.1	0.8369	0.8352	-0.0017
Greedy, 4-gram block, rp 1.2	0.8372	0.8363	-0.0009
MBR, ROUGE-L utility, 14 candidates	0.8393	0.8379	-0.0014
MBR, BERTScore utility, 14 candidates	**0.8419**	**0.8407**	-0.0012

Table 15: Selected Phase 1 pseudo-validation results (300 prompts) with the secondary metrics; rep-4 = share of repeated word 4-grams, dist-2 = share of distinct bigrams.

Strategy	BERTScore	ROUGE-L	BLEU-4	rep-4	dist-2
Greedy, no blocking	0.8048	0.1898	0.0359	0.693	0.267
Greedy, 3-gram block	0.8351	0.2046	0.0576	0.000	0.807
Greedy, 6-gram block	0.8295	0.2055	0.0529	0.174	0.592
Greedy, 4-gram, rp 1.1	0.8369	0.1995	0.0634	0.000	0.831
Greedy, 4-gram, rp 1.2	0.8372	0.1865	0.0533	0.000	0.931
Top-p T=0.9, 4-gram	0.8318	0.1891	0.0406	0.000	0.887
Beam 4, 3-gram block	0.8353	**0.2069**	**0.0673**	0.000	0.872
Contrastive α=0.6	0.8263	0.1934	0.0412	0.029	0.869
MBR, utility ROUGE-L	0.8393	0.2057	0.0655	—	—
MBR, utility BERTScore	**0.8419**	0.2041	0.0636	—	—

Table 16: Phase 1 training runs (tokens per optimizer step, tokens seen, peak learning rate).

Model	Tokens/step	Tokens	Peak LR
M1	32,768	200M	3×10^{-4}
M2	16,384	147M	3×10^{-4}
M3	16,384	98M	1×10^{-3}
M4	65,536	214M	6×10^{-4}
M5	65,536	—	6×10^{-4}



(a) Codabench scores in the order the changes were made (blue: architectures, orange: decoding).

(b) Pseudo-validation BERTScore of all 28 decoding settings on Model 4.

Figure 6: Phase 1 progression and decoding study.

D Phase 2 details

Table 17: Phase 2 runs: strict macro-F1 / precision / recall on the 10 validation documents at the best epoch, and Codabench where submitted.

Run	Model	Method	Output layer	Trained	Ep.	Val F1	P	R	Codabench
`m4_lorma`	M4	LoRMA	softmax	1.13M	4	0.3202	0.305	0.368	0.3442
`m2_lora`	M2	LoRA	softmax	1.04M	6	0.2899	0.269	0.325	0.3416
`m3_full`	M3	full	softmax	68.4M	8	0.1906	0.204	0.187	
`m3_bitfit`	M3	BitFit	softmax	0.08M	7	0.1149	0.146	0.099	
`m4_lorma_vit`	M4	LoRMA	constrained Viterbi	1.13M	6	0.4131	0.422	0.419	
`m4_lorma_crf`	M4	LoRMA	CRF	1.13M	8	0.4502	0.531	0.415	
`m4_full`	M4	full	softmax	101.4M	7	0.3450	0.335	0.372	
`m4_kd_student`	M4 (4 layers)	KD	softmax	63.6M	8	0.2841	0.259	0.322	
`m4_small_nokd`	M4 (4 layers)	full (control)	softmax	63.6M	7	0.2695	0.241	0.314	
`m4_lorma_crf_pre`	M4	LoRMA	CRF + preamble 256	1.13M	8	0.4561	0.555	0.419	0.4754*
`m5_lorma_crf_pre`	M5	LoRMA	CRF + preamble 256	1.13M	3	**0.4797**	0.517	0.470	**0.5122**†

Codabench of the run's own submission; *+ propagate; †+ vote + propagate. `m4_lorma_crf` + vote + propagate: validation 0.4813, Codabench 0.4715.

Table 18: Phase 2: gold → tags → spans round trip (strict macro-F1 ceiling) and validation strict macro-F1 of the document-level rules.

Trimming	train	val	Base	none	+vote	+prop.	+both
none (raw token spans)	0.605	0.650	`m4_lorma_crf`	0.4502	0.4591	0.4739	0.4813
basic (edge punctuation)	0.947	0.932	`m4_lorma_vit`	0.4131	0.4153	0.4058	0.4079
smart (abbrev./bracket/possessive)	0.979	0.973	`m4_full`	0.3450	0.3475	0.3246	0.3275

Table 19: Phase 2 training documents: share whose first mention of a label lies within the first P tokens, and share of all mentions beyond token 1,024 (used to choose the preamble length).

	P=64	128	256	512	768	>1024
APP	90%	95%	98%	100%	100%	46%
RESP	81%	90%	94%	97%	98%	55%
JUDGE	62%	76%	77%	82%	83%	53%
A.COUNSEL	18%	24%	39%	43%	51%	53%
R.COUNSEL	11%	23%	36%	40%	49%	63%

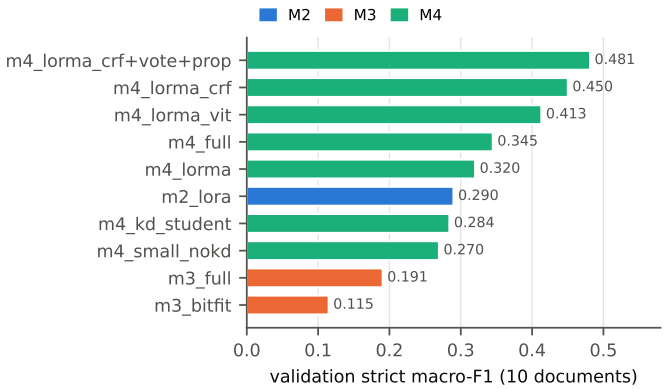


Figure 7: Phase 2 validation strict macro-F1 of the first two sessions' runs.

E Phase 3 details

Table 20: Phase 3 runs: dev macro-F1 and per-domain F1 (CL / IT); all runs read document windows.

Run	Method / head	Trained	Dev	CL / IT
`m2_lora`	LoRA / linear	1.03M	0.415	.361 / .416
`m2_rslora`	rsLoRA / linear	1.03M	0.431	.365 / .447
`m2_lorma`	LoRMA / linear	0.95M	0.424	.341 / .437
`m3`	full / linear	68.3M	0.205	.182 / .209
`m4_lora`	LoRA / linear	1.10M	0.435	.368 / .413
`m4_rslora`	rsLoRA / linear	1.10M	0.433	.351 / .457
`m4_lorma`	LoRMA / linear	1.12M	0.438	.359 / .450
`m4_full`	full / linear	101M	0.437	.343 / .443
`m5_lora`	LoRA / linear	1.10M	0.441	.354 / .418
`m5_rslora`	rsLoRA / linear	1.10M	0.450	.381 / .436
`m5_lorma`	LoRMA / linear	1.12M	0.431	.382 / .406
`m5_full`	full / linear	101M	0.442	.374 / .447
`m5_lora_crf`	LoRA / CRF	1.10M	**0.469**	.398 / .446
`m5_rslora_crf`	rsLoRA / CRF	1.10M	0.453	.383 / .435
`m5_lorma_crf`	LoRMA / CRF	1.12M	0.427	.368 / .425
`m5_full_crf`	full / CRF	101M	0.455	.378 / .461
`m5_kd_student`	KD 4 layers / linear	63.6M	0.347	.265 / .390

Table 21: Phase 3 decoding strategies: dev macro-F1 (Codabench in brackets where submitted).

Base	argmax / own CRF	prior	prior + Viterbi	+ position prior
`m2_ens`	0.425	0.443	0.439 (0.495)	0.422
`m3`	0.205	0.192	0.350 (0.334)	0.353
`m4_ens`	0.454	0.421	0.441 (0.525)	0.453
`m5_ens`	0.448	0.458	0.463 (0.562)	0.465
`m5_full_crf`	0.453	0.452	0.472 (0.543)	0.496 (0.559)
`m5_crf_ens`	0.453 (0.504)	0.457	0.471 (**0.571**)	0.474 (0.556)

Expected-F1 decoding of `m5_crf_ens` (coordinate ascent on HMM posteriors): dev 0.469, Codabench 0.5657.

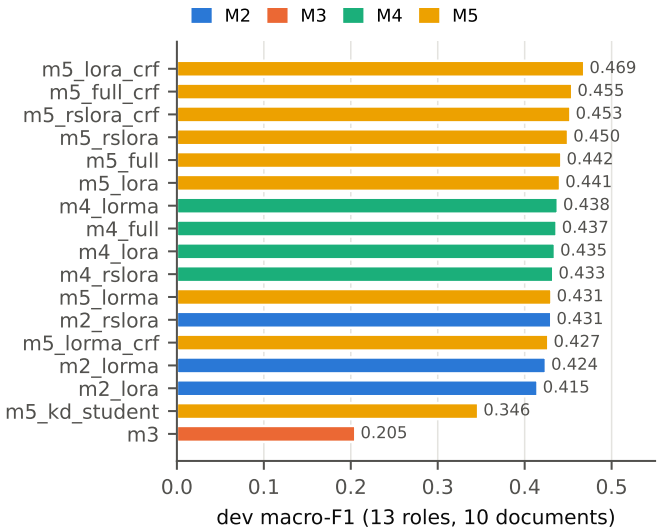


Figure 8: Phase 3 dev macro-F1 of all runs (argmax or the run's own CRF decoding).

F Phase 4 details

Table 22: Phase 4 runs: dev macro-F1 on the 994 development cases (argmax, reading the section the run was trained on); Min. = training minutes on one T4.

Run	Method	Reads	Trained	Cases	Ep.	Dev F1	Min.
`m5_rslora_alldocs`	rslora	tail 1×512	1.28M	31.7k	3	0.7087	186
`m5_lora_alldocs`	lora	tail 1×512	1.28M	31.7k	3	0.6980	177
`m5_full_alldocs`	full	tail 1×512	101.5M	31.7k	3	0.6874	201
`m2_lorma`	lorma	tail 1×512	1.14M	10.0k	3	0.6831	59
`m5_rslora`	rslora	tail 1×512	1.28M	10.0k	3	0.6615	65
`m2_lora`	lora	tail 1×512	1.22M	10.0k	2	0.6609	58
`m5_lorma_hier4`	lorma	tail 4×512	1.30M	10.0k	3	0.6566	192
`m5_full`	full	tail 1×512	101.5M	10.0k	1	0.6528	69
`m5_lorma_alldocs`	lorma	tail 1×512	1.30M	31.7k	3	0.6519	175
`m5_kd_student`	kd	tail 1×512	63.8M	10.0k	2	0.6503	26
`m4_rslora`	rslora	tail 1×512	1.28M	10.0k	1	0.6415	65
`m5_lora`	lora	tail 1×512	1.28M	10.0k	3	0.6398	65
`m4_full`	full	tail 1×512	101.5M	10.0k	2	0.6389	69
`m2_rslora`	rslora	tail 1×512	1.22M	10.0k	3	0.6384	59
`m4_lorma`	lorma	tail 1×512	1.30M	10.0k	1	0.6318	64
`m5_lorma`	lorma	tail 1×512	1.30M	10.0k	3	0.6233	64
`m4_lora`	lora	tail 1×512	1.28M	10.0k	1	0.6224	65
`m3`	full	tail 1×512	68.8M	10.0k	3	0.5901	11
`m5_lorma_head`	lorma	head 1×512	1.30M	10.0k	3	0.5602	58

Table 23: Phase 4 submissions: dev, 2-fold CV and Codabench macro-F1.

Submission	Dev	CV	Codabench
`m5_rslora`, both readings, dev threshold	0.7003	0.6924	0.6941
`m2_lorma`, both readings, training-prior threshold	0.7188	(fixed rule)	0.6889
`m5_rslora_alldocs`, last 8×512, dev threshold	0.7665	0.7537	0.7374
`m5_alldocs_ens`, last 8×512, dev threshold	0.7612	0.7521	**0.7609**
`m5_alldocs_ens`, both readings, dev threshold	0.7594	0.7484	0.7523

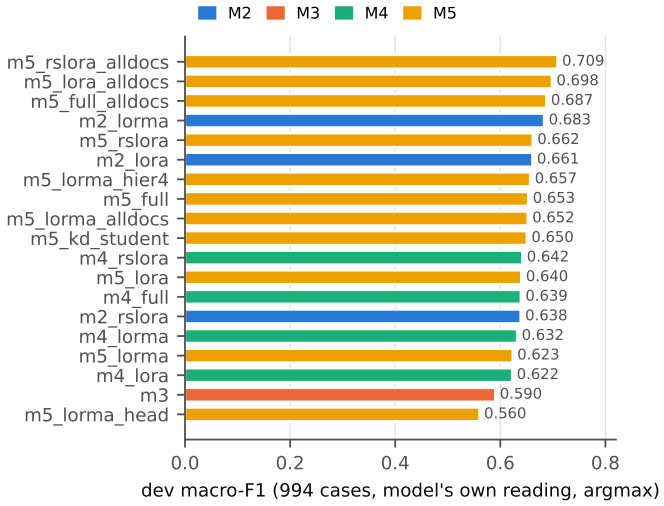


Figure 9: Phase 4 dev macro-F1 of every run.

G Phase 5 details

Table 24: Phase 5 runs: full-dev macro-F1 and micro-F1 with one global threshold (Thr.).

Run	Method	Reads	Trained	Docs	Ep.	Dev F1	Micro	Thr.
`m4_full`	full	head 4×512	101.6M	15k	3	0.3535	0.481	0.18
`m5_full`	full	head 4×512	101.6M	15k	3	0.3531	0.480	0.18
`m5_lorma_head1_alldocs`	lorma	head 1×512	1.38M	42k	3	0.3490	0.470	0.16
`m5_kd_student`	kd	head 4×512	63.8M	15k	3	0.3460	0.466	0.14
`m4_lorma`	lorma	head 4×512	1.38M	15k	3	0.3414	0.468	0.16
`m5_partial`	partial	head 4×512	31.7M	15k	3	0.3364	0.466	0.16
`m5_lorma_sqrtpw`	lorma	head 4×512	1.38M	15k	3	0.3355	0.439	0.52
`m5_lorma`	lorma	head 4×512	1.38M	15k	3	0.3292	0.459	0.16
`m2_lorma`	lorma	head 4×512	1.22M	15k	3	0.3278	0.448	0.14
`m3`	full	head 4×512	69.0M	15k	3	0.2105	0.346	0.08

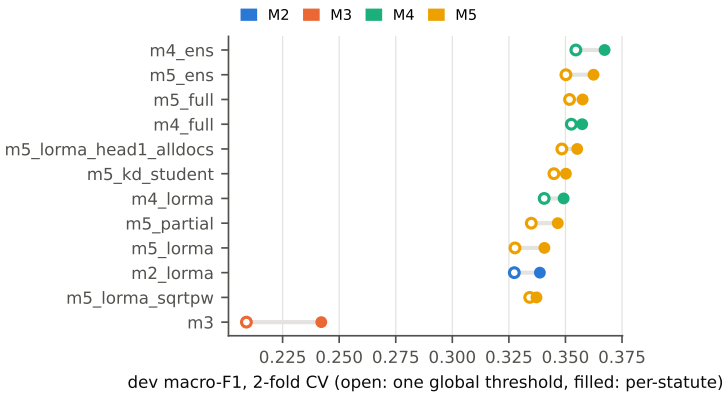


Figure 10: Phase 5 dev macro-F1 (2-fold CV) with one global threshold (open) and per-statute thresholds (filled).

H Phase 6 details

Table 25: Phase 6 runs: held-out loss and the 100 generated held-out summaries (BERTScore F1, ROUGE-1/L, mean length in words; references average 852 words).

Run	Method	Trained	Ep.	Val loss	BERTScore	R-1	R-L	Words	Min.
`m4_lora`	lora	1.09M	2	2.570	0.8378	0.447	0.236	535	122
`m2_houlsby`	houlsby	2.01M	2	2.519	0.8377	0.459	0.238	664	113
`m5_lorma`	lorma	1.11M	2	2.685	0.8361	0.442	0.237	630	121
`m5_kd_student`	kd	63.6M	2	3.798	0.8232	0.407	0.224	566	103
`m3_full`	full	91.7M	0.94	4.444	0.8033	0.319	0.174	747	81

Table 26: Phase 6 decoding variants and ensembles on the held-out set, and Codabench where submitted.

System	Held-out BERTScore	ROUGE-1	Codabench
`m4_lora`, repetition penalty 1.15	0.8423	—	**0.8381**
`m4_lora`, 3-gram block	0.8414	—	0.8380
MBR consensus over all runs	0.8398	0.476	
MBR over `m4_lora` variants	0.8376	—	
MBR over `m2_houlsby` variants	0.8371	—	

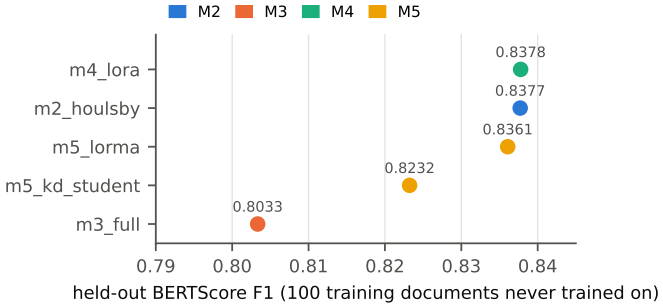


Figure 11: Phase 6 held-out BERTScore F1 of every run (greedy, no-repeat 4-grams).

I Phase 7 details

Table 27: Phase 7 runs: dev Micro-F1@7 of the untrained Phase 1 encoder (0-shot) and at the best epoch, recall in the top 100; raw cosine ranking with the date and self rules. The two Model 2 runs did not run (dataset not attached).

Run	Method	Layers	Trained	Ep.	0-shot	Dev F1@7	R@100	Min.
`m5_full`	full	10/10	101.3M	5	0.2424	0.3146	0.680	35
`m4_full`	full	10/10	101.3M	6	0.2457	0.3102	0.700	34
`m5_lorma`	lorma	10/10	1.11M	6	0.2424	0.3091	0.692	37
`m5_kd_student`	kd	4/10	63.6M	5	0.2379	0.2913	0.690	14
`m3_full`	full	2/2	68.3M	4	0.1467	0.2724	0.660	16
`m4_ia3`	ia3	10/10	0.03M	4	0.2457	0.2690	0.668	33

Table 28: Phase 7 post-processing: raw ranking, dev-selected setting (Dev), its 2-fold CV estimate, the same without the query penalty ($^-$), and Codabench (– = not submitted). M5 LoRMA reading 4 chunks: `head4` 0.3302, `spread4` 0.3413 raw.

Base	Raw	Dev	CV	Dev$^-$	CV$^-$	LB	LB$^-$
`bm25`	0.1857	0.4325	0.4264	0.4269	0.4249	0.4306	0.4318
`m3_full`	0.2724	0.4325	0.4263	0.4258	0.4200	–	–
`m4_full`	0.3102	0.4336	0.4280	0.4280	0.4228	0.4253	–
`m4_ia3`	0.2690	0.4336	0.4257	0.4280	0.4236	–	–
`m5_full`	0.3146	0.4369	0.4311	0.4280	0.4240	0.4288	–
`m5_lorma`	0.3091	0.4369	0.4308	0.4258	0.4202	–	–
`m5_lorma__head1`	0.2779	0.4369	0.4331	0.4269	0.4223	0.4259	–
`m5_lorma__spread4`	0.3413	0.4380	0.4317	0.4269	0.4200	0.4300	–
`m5_kd_student`	0.2913	0.4314	0.4251	0.4269	0.4226	–	–
`all_ens`	0.3191	0.4347	0.4284	0.4291	0.4245	0.4300	0.4318

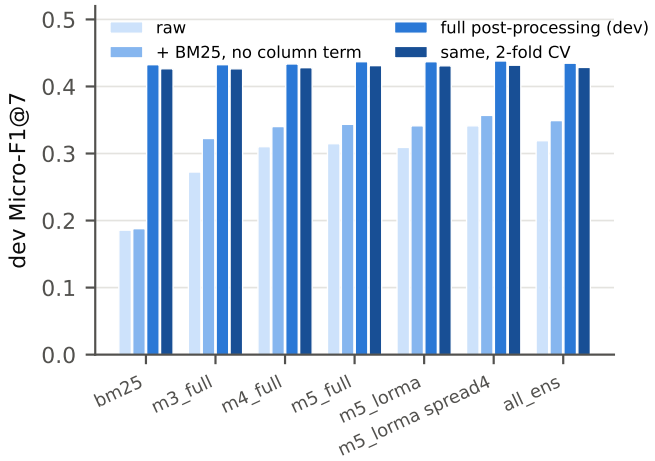


Figure 12: Phase 7 post-processing steps on dev.

J Implementation details

Parallel CRF partition function (Phase 2). The linear-chain CRF scores a tag sequence as $s(y) = start[y_0] + \sum_t em[t, y_t] + \sum_{t \geq 1} trans[y_{t-1}, y_t] + end[y_{n-1}]$ and is trained on $\log Z - s(y_{\text{gold}})$ per tagged token, with invalid BIO transitions fixed at -10^4. The sequential forward recursion for $\log Z$ took 3.8 s per batch of four 1,024-token windows. Writing each step as a 25×25 matrix $M_t[i, j] = trans[i, j] + em[t, j]$ in the log semiring (where "multiplication" is log-sum-exp of sums) and multiplying neighbours pairwise needs $\log_2 1024 = 10$ parallel rounds: 0.036 s per batch, with results equal to the sequential recursion and to brute-force enumeration on short sequences. The CRF adds 675 parameters.

Preamble context (Phase 2). With `preamble_tokens`$= P$, every window not starting at the document start receives the document's first P tokens in front of its content (content shrinks to $1024 - P$); these positions carry no loss, no distillation signal and no predictions. $P = 256$ covers the first appellant/respondent mention in 98%/94% of training documents (Table 19); $P = 512$ would remove the window overlap.

Decoding from training statistics (Phase 3). With per-sentence posteriors $p(y \mid x_t)$, the prior strategy decodes $\arg\max_y \log p(y \mid x_t) - \log \pi(y)$ with the training role frequencies π; prior + Viterbi finds the role sequence maximising $\sum_t [\log p(y_t \mid x_t) - \log \pi(y_t)] + \sum_t \log A(y_{t-1}, y_t)$ with training transition probabilities A; the position prior adds $\log p(y \mid b_t) - \log \pi(y)$ for 10 relative-position bins b_t. CRF runs are first converted to per-sentence posteriors by forward–backward with their learned transitions.

Gradient caching (Phase 7). A batch of 16 queries with one positive and one hard negative each holds about 48 cases of 1,024 tokens. I (i) encode all cases without gradients, (ii) compute the InfoNCE loss and its gradient with respect to the embedding matrix, and (iii) re-encode each sub-batch of $\leq$4,096 tokens with gradients and back-propagate the cached embedding gradient into it. Parameter gradients matched direct back-propagation to a relative 10^{-6}, and a single batch could be over-fitted (loss 2.65 $\rightarrow$ 0.0016). The KD student matches the teacher's in-batch similarity distributions (teacher embeddings of the training cases are pre-computed once).

K Code and reproducibility

The repository https://github.com/D3bug1t/LLM-Pretraining-PostTraining-Extrinsic-Instrinsic-Evals (and the `code/` folder of the submitted archive) contains the phase notebooks, post-processing scripts and official evaluation scripts, with a README per phase. The Phase 2–7 figures and appendix tables are generated from saved results by `make_figures.py` and `data_stats.py`. The separate course-required LLM analysis is `llm_based_results_and_data_analysis.pdf`.